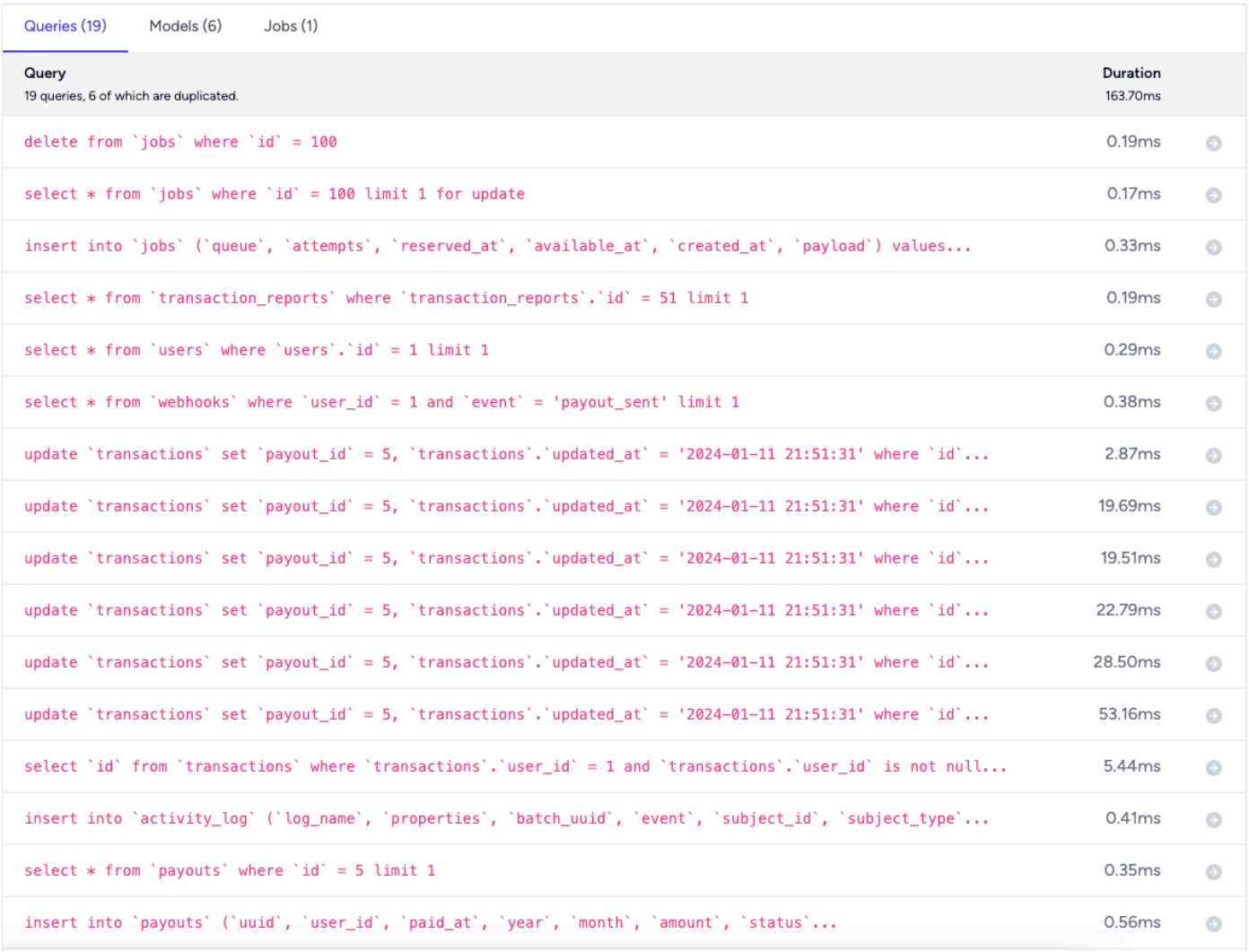
```php
$transactionIds = $query->select('id')->pluck('id');

$chunks = $transactionIds->chunk(1000);

foreach ($chunks as $chunk) {
  Transaction::query()
    ->whereIn('id', $chunk)
    ->update(['payout_id' => $payout->id]);
}
```

Now it's only updating 1,000 transactions at a time. So it results in smaller and faster queries:



Queries (19) | Models (6) | Jobs (1)

Query 19 queries, 6 of which are duplicated.	Duration 163.70ms
delete from `jobs` where `id` = 100	0.19ms
select * from `jobs` where `id` = 100 limit 1 for update	0.17ms
insert into `jobs` (`queue`, `attempts`, `reserved_at`, `available_at`, `created_at`, `payload`) values...	0.33ms
select * from `transaction_reports` where `transaction_reports`.`id` = 51 limit 1	0.19ms
select * from `users` where `users`.`id` = 1 limit 1	0.29ms
select * from `webhooks` where `user_id` = 1 and `event` = 'payout_sent' limit 1	0.38ms
update `transactions` set `payout_id` = 5, `transactions`.`updated_at` = '2024-01-11 21:51:31' where `id`...	2.87ms
update `transactions` set `payout_id` = 5, `transactions`.`updated_at` = '2024-01-11 21:51:31' where `id`...	19.69ms
update `transactions` set `payout_id` = 5, `transactions`.`updated_at` = '2024-01-11 21:51:31' where `id`...	19.51ms
update `transactions` set `payout_id` = 5, `transactions`.`updated_at` = '2024-01-11 21:51:31' where `id`...	22.79ms
update `transactions` set `payout_id` = 5, `transactions`.`updated_at` = '2024-01-11 21:51:31' where `id`...	28.50ms
update `transactions` set `payout_id` = 5, `transactions`.`updated_at` = '2024-01-11 21:51:31' where `id`...	53.16ms
select `id` from `transactions` where `transactions`.`user_id` = 1 and `transactions`.`user_id` is not null...	5.44ms
insert into `activity_log` (`log_name`, `properties`, `batch_uuid`, `event`, `subject_id`, `subject_type`...	0.41ms
select * from `payouts` where `id` = 5 limit 1	0.35ms
insert into `payouts` (`uuid`, `user_id`, `paid_at`, `year`, `month`, `amount`, `status`...	0.56ms

Of course, we have more queries but they take less time and probably less memory and less overhead for MySQL. In general, it's always a good idea to split big stuff into smaller pieces. It can be a huge class, a long method, a big monolith, or a pretty long update query.

All right, these were the two most obvious performance problems in the application and we successfully solved them. If you want to learn more about performance check out my upcoming book Performance with Laravel.